Explainable SYN/UDP Flood Detection at the IoT Edge: A Lightweight Fuzzy Decision Tree for Transport-Layer Network Forensics

Abhishek Anand
Dept. of CSE
Indian Institute of Information Technology Sri City
Chittoor, Andhra Pradesh, India

Neha Agarwal
Dept. of CSE
Indian Institute of Information Technology Sri City
Chittoor, Andhra Pradesh, India

Pavan Kumar B N
Dept. of CSE
Indian Institute of Information Technology Sri City
Chittoor, Andhra Pradesh, India

***Abstract*—Internet-of-Things (IoT) networks are a prime target of transport-layer denial-of-service floods, yet IoT network forensics is still largely manual: analysts read Wireshark I/O graphs and apply display filters to count SYN and SYN-ACK packets. Learning-based detectors automate this step but are usually too heavy for edge hardware and too opaque for an analyst who must justify blocking a host. This paper presents FUZZYFORENSIOT, an explainable edge intrusion-detection and forensic-triage framework. It turns the transport-layer indicators that manual IoT forensics relies on (SYN/ACK imbalance, half-open connections, packet-rate bursts, source and port dispersion) into graded, human-readable verdicts. Twenty-two rolling-window features are mapped to 30 data-driven triangular membership grades and eight direction-gated expert rules. A cost-complexity-pruned decision tree is then trained only on this fuzzy space and fused with the rule evidence. The fused threat score drives a graduated ALLOW/THROTTLE/BLOCK response on a Raspberry Pi. We also contribute two evaluation-hygiene steps for packet-window datasets. Direction-aware relabeling showed that 49.1% of weakly labelled attack packets were in fact the victim's own replies. A buffered chronological split removes the leakage caused by overlapping rolling windows, which otherwise inflates accuracy by about two points. On a 194,792-packet lab capture with a leakage-free hold-out of 38,747 packets, the deployed 89-node (9.1 KB) fuzzy decision tree reaches 87.74% accuracy and an F1-score of 0.456. It outperforms an identically trained plain decision tree (86.39%, 0.438) and a $125\times$ larger deep multilayer perceptron (86.51%, 0.440), and raises 583 fewer false alarms than the plain tree. Every verdict comes with IF-THEN evidence that can be placed directly in a forensic report.**

***Index Terms*—IoT security, network forensics, SYN flood, UDP flood, fuzzy decision tree, explainable intrusion detection, edge computing, Raspberry Pi**

I. INTRODUCTION

The Internet of Things now spans smart homes, medical devices and industrial control. Tens of billions of connected objects exchange data over standard Internet protocols [1]. Most of these devices have little memory, little CPU and rarely receive security updates. This makes them both easy victims and easy recruits for denial-of-service (DoS) campaigns. The Mirai botnet showed the scale of the problem by turning around 600,000 compromised IoT devices into record-breaking floods [2], [3].

Transport-layer floods remain the workhorse of such campaigns. A TCP SYN flood abuses the three-way handshake. It leaves the victim with a backlog of half-open connections until legitimate clients are refused [4], [5]. A UDP flood saturates bandwidth and CPU with datagrams, and the victim answers closed ports with ICMP *destination unreachable* messages [6]. Naing *et al.* report that over 90% of DoS attacks exploit TCP, with SYN flooding the most common form [1].

IoT forensics is commonly organised into device-, network- and cloud-level investigations [7]. At the network level, the state of practice is manual. Naing *et al.* [1] investigated SYN and UDP floods against two smart-home devices by applying Wireshark filters such as `tcp.flags.syn==1 && tcp.flags.ack==0`. They concluded that an attack had occurred because 5,636 SYN packets received only 225 SYN-ACK replies. Khaing *et al.* [6] identified UDP floods from bursts of ICMP host-unreachable packets and UDP datagrams sent to port 80. Both studies identify useful forensic indicators, but a human must apply the filters, read the graphs and decide. The analysis is reactive, does not scale to continuous monitoring, and gives a binary judgement with no measure of confidence.

On the reasoning side, Ye [8] argued that digital evidence is inherently "specious" (neither fully guilty nor fully innocent). Ye proposed fuzzy decision trees (FDTs) built with Fuzzy-ID3 as a reasoning engine for proactive network forensics, where every root-to-leaf path becomes a fuzzy rule that a court can read. However, that work is conceptual. No membership functions are defined for network metrics, no dataset is used and no accuracy is reported.

At the edge, a detector must meet four requirements at once. It must be *lightweight* (kilobytes, microseconds), *accurate*, *explainable*, and *forensically sound* (it must attribute evidence to the attacker rather than the victim). Crisp thresholds fail in the grey zone where legitimate congestion and low-rate floods look alike. Deep models detect well [9], [10], but they consume megabytes and behave as black boxes [11]. Classical rule-based systems are interpretable but rigid and sensitive to hand-chosen constants [12].

This paper closes the gap between manual transport-layer

forensics [1], [6] and fuzzy forensic reasoning [8]. Our contributions are:

1) **FUZZYFORENSIOT**, an end-to-end framework that turns raw IoT packet streams into graded flood verdicts using a fuzzy rule layer and a lightweight decision tree, together with human-readable evidence.
2) A **direction-gated fuzzy rule base** with **data-driven membership placement**, which needs no hand-tuned constants, plus a rule–tree fusion score that drives a graduated response.
3) **Evaluation hygiene for packet-window data**. Direction-aware relabeling corrects 49.1% of weak attack labels. A buffered chronological split removes rolling-window leakage that otherwise inflates baseline accuracy by 2.06 points.
4) A **controlled comparison** under one shared training protocol. The 9.1 KB fuzzy tree beats an identically trained plain tree and a 1.14 MB deep MLP on accuracy, precision and F1. Ablations isolate the contributions of the rules, the tree and the fusion step.
5) A **Raspberry Pi deployment** with live capture, ALLOW/THROTTLE/BLOCK mitigation with time-to-live (TTL) auto-release, management safety rails, a forensic event log and an explanation dashboard.

Section II reviews related work. Section III formalises the problem. Section IV describes how the dataset was generated and conditioned. Section V presents the fuzzy reasoning pipeline, and Section VI the edge deployment. Sections VII and VIII report the experiments. Section IX discusses limitations, and Section X concludes.

II. Background and Related Work

A. Transport-Layer Flooding Attacks

In a legitimate TCP handshake the client sends SYN, the server answers SYN-ACK and allocates state, and the client completes with ACK (Fig. 2a). A SYN flood sends SYNs, often from spoofed addresses, and never completes the handshake. Each SYN-ACK is therefore sent into the void and the victim's backlog of half-open connections fills up (Fig. 2b) [4], [5]. The forensic signatures are a large SYN-to-SYN-ACK imbalance, a growing number of half-open connections, short SYN inter-arrival times and abnormal TCP fields. Naing *et al.* observed flood SYNs with zero payload and a window size of 256 [1]. A UDP flood sprays datagrams at random or fixed ports. Closed ports trigger ICMP destination-unreachable replies, and UDP traffic towards TCP-only service ports (e.g., port 80) is itself anomalous [6].

B. IoT Network Forensics

Stoyanova *et al.* [7] survey IoT forensics and highlight open problems in data heterogeneity, evidence preservation and legal admissibility. Network forensics follows a process model of preparation, detection, incident response, collection, preservation, examination, analysis, investigation and presentation [1]. The two transport-layer studies that motivate this work [1], [6] carry out the detection and analysis stages by hand in Wireshark. Naing *et al.* analysed a capture of more than 1 GB (187,060 records) from an SKT NUGU speaker and an EZVIZ camera. They identified SYN floods from the SYN versus SYN-ACK counts, I/O-graph spikes, anomalous packet-length bins (40–2,599 bytes), and out-of-order and retransmitted segments. Khaing *et al.* attributed UDP floods to four attacker addresses by filtering ICMP host-unreachable packets and UDP traffic to port 80. Koroniotis *et al.* released Bot-IoT specifically for network-forensic analytics [13]. None of these works produces automated, confidence-graded verdicts.

C. Fuzzy Logic and Fuzzy Decision Trees

Fuzzy sets [14] replace the crisp 0/1 indicator with a membership degree $\mu \in [0, 1]$. This lets a value such as a SYN rate be "somewhat high" and "somewhat medium" at the same time. Decision-tree induction (ID3 [15], CART [16]) recursively partitions data by an impurity criterion. Each root-to-leaf path is a conjunctive rule. Fuzzy-ID3 [17] and related FDT methods [18] combine both ideas: attributes are fuzzified and a fuzzy entropy guides the splits. Ye [8] applied this reasoning to network forensics. That work describes a four-step FDT construction (fuzzify the attributes, create the root, choose the best fuzzy split, recurse), extracts one rule per path, and argues for proactive rather than event-driven evidence collection. It contains no experiments.

D. Machine-Learning and Fuzzy IDS for IoT DDoS

Doshi *et al.* [9] showed that lightweight packet and flow features separate consumer-IoT DDoS traffic well. N-BaIoT [10] uses per-device deep autoencoders. Dimolianis *et al.* [19] classify SYN floods with machine learning and mitigate them with programmable data-plane filters. Fuzzy approaches include an FIS-based DDoS IDS evaluated on IoT-23 [20], a hybrid ML–fuzzy-cognitive-map detector aimed at resource-constrained devices [21], and FLSec-RPL, a fuzzy IDS with a blocking stage for RPL routing attacks [22]. Large IoT attack benchmarks such as CICIoT2023 [23] have made these comparisons more systematic.

E. Research Gap

Table I positions this work. The forensic studies [1], [6] provide validated transport-layer indicators but no automation. The FDT forensics model [8] provides the reasoning paradigm but no implementation. Fuzzy and ML IDS works automate detection but do not combine KB-scale edge deployment, per-verdict forensic explanations and attacker-direction attribution in a single system. To our knowledge, no prior system defines membership functions for IoT transport-layer flood metrics and trains an FDT-style model on real IoT packet captures to emit confidence-graded, court-readable rules.

III. Problem Formulation and Design Goals

Let $\mathcal{P} = \langle p_1, p_2, \dots \rangle$ be the packet stream observed at an IoT edge node. Each packet p_i has a timestamp t_i, source and destination addresses and ports, a protocol, a length ℓ_i and

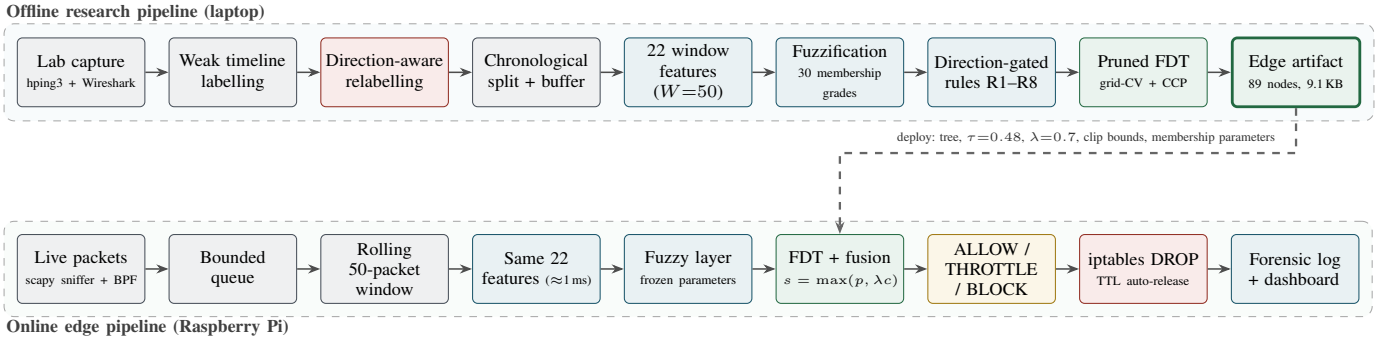


Fig. 1. Overview of FUZZYFORENSIOT. The offline pipeline conditions the capture, builds the fuzzy feature space and trains a pruned fuzzy decision tree. The edge pipeline reproduces the same feature and fuzzy transformations with frozen parameters, scores every 50-packet window and applies a graduated, reversible response with logged evidence.

TABLE I
POSITIONING AGAINST THE BASE PAPERS AND RELATED IOT DDOS DETECTORS (✓ = PROVIDED, ∼ = PARTIAL OR CLAIMED, × = ABSENT, – = NOT REPORTED)

Work	Target attack	Automated	Fuzzy reasoning	Empirical data	Edge-deployable	Per-verdict explanation	Mitigation
Naing *et al.* [1] (base)	TCP-SYN, UDP flood	×	×	✓	×	∼ (analyst)	×
Khaing *et al.* [6] (base)	UDP flood	×	×	✓	×	∼ (analyst)	×
Ye [8] (base)	Generic intrusion	∼ (conceptual)	✓	×	×	✓ (conceptual)	×
Dimolianis *et al.* [19]	TCP-SYN flood	✓	×	✓	✓ (data plane)	–	✓
Hoang *et al.* [20]	DDoS (IoT-23)	✓	✓ (FIS)	✓	∼	∼	–
Keshavamurthy & Kulkarni [21]	DDoS	✓	✓ (FCM)	✓	∼	∼	–
Kim *et al.* [22]	RPL DIO suppression	✓	✓	∼ (simulation)	∼	∼	✓
FUZZYFORENSIOT (this work)	**TCP-SYN, UDP flood**	✓	✓ (rules + FDT)	✓	✓ (Raspberry Pi)	✓ (rules + path)	✓ (graduated)

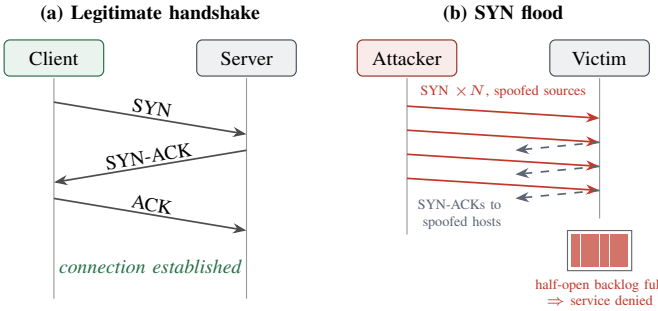


Fig. 2. (a) A legitimate TCP handshake. (b) A SYN flood: unanswered SYN-ACKs leave half-open entries that exhaust the victim's backlog, producing the SYN/SYN-ACK imbalance observed manually in [1].

TCP flags. For every packet we form the backward-looking window

$$\mathcal{W}_i = \langle p_{i-W+1}, \ldots, p_i \rangle, \qquad W = 50, \tag{1}$$

and a feature vector $\mathbf{x}_i = \phi(\mathcal{W}_i) \in \mathbb{R}^{22}$. The label is $y_i = 1$ if and only if p_i flows from an attacker to the victim during an attack phase. The victim's replies are deliberately labelled 0 (Section IV-C). We seek a decision function

$$f : \mathbf{x}_i \mapsto (\hat{y}_i,\ s_i,\ \mathcal{E}_i), \tag{2}$$

which returns a verdict $\hat{y}_i$, a threat score $s_i \in [0, 1]$ and an explanation $\mathcal{E}_i$ made of named rule activations and the decision path. The design goals are:

- **G1 Lightweight:** kilobyte-scale model with microsecond inference on a single CPU core.
- **G2 Accurate:** at least as accurate as a deep baseline under leakage-free evaluation.
- **G3 Explainable:** every verdict maps to named forensic indicators.
- **G4 Forensically sound:** evidence is attributed to attacker-direction traffic, and the response is graded, reversible and logged.

IV. DATASET GENERATION AND CONDITIONING

A. Testbed and Attack Generation

Fig. 3 shows the testbed. A Raspberry Pi acts as the target IoT edge node on an isolated lab Wi-Fi network. An attacker laptop runs `hping3` [24] with a scripted attack cycle: a 20 s SYN flood to port 80 (`-S -p 80 -i u2000 --rand-source`, nominally 500 pps), 10 s of silence, and a 20 s UDP flood to port 53 (`-2 -p 53`). Each phase boundary is written with its wall-clock timestamp to `labels_map.txt`. Traffic is captured with Wireshark/tcpdump and exported to CSV with the fields time, source, destination, protocol, length and info. Ordinary devices on the same network contribute realistic background traffic (TCP, QUIC, DNS, mDNS, ARP, ICMPv6 and TLS). Table II summarises the capture.

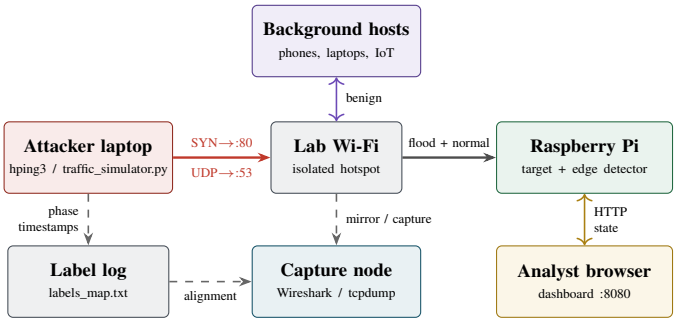


Fig. 3. Lab testbed. Floods and benign traffic reach the Raspberry Pi through an isolated Wi-Fi network. The capture is aligned with the attack-phase timestamps logged by the generator, and the Pi serves the live dashboard.

TABLE II
LAB CAPTURE STATISTICS

Property	Value
Packets captured	194,792
Duration	2,283 s (≈38 min)
TCP / UDP / QUIC / ICMP share	73.8 / 16.5 / 5.2 / 2.6%
SYN-flood phase (weak label)	20.1 s, 20,572 pkts, 1,025 pps
Second flood phase (weak label)	20.0 s, 8,371 pkts, 419 pps
Weakly labelled attack packets	28,943 (14.86%)
Victim replies relabelled normal	14,208 (49.1%)
Unrelated background relabelled	215
Final attack / normal packets	**14,520 (7.45%) / 180,272**
Train / test / buffer (dropped)	155,835 / 38,747 / 210
Attack ratio train / test	7.38% / 7.64%

B. Weak Timeline Labelling

The generator logs phase boundaries in wall-clock time, while the capture uses relative timestamps. The labeller therefore aligns the two timelines. It uses the capture's start epoch when available, and otherwise searches for the offset (on a 0.1 s grid) that maximises the contrast between the packet rates inside and outside the attack intervals. Every packet inside an attack interval receives the label 1. This produces 28,943 weakly labelled attack packets.

C. Direction-Aware Relabelling

Window-based labels are cheap but noisy. During a SYN flood, a large share of the packets inside the attack window are the *victim's own* SYN-ACK and RST replies. A model trained to call these "attack" learns to flag the victim, which caps its recall and, in deployment, triggers alarms on the edge node's own traffic. Algorithm 1 infers the victim of each attack window and relabels its replies and any unrelated background chatter as normal. This step reverted 14,208 victim replies (49.1%) and 215 background packets, leaving 14,520 attacker-to-victim packets. In our development experiments, removing this label noise raised the ROC-AUC from about 0.79 to about 0.90.

Algorithm 1 Direction-aware relabelling

Input: weakly labelled packets; contiguous attack windows $\{\mathcal{I}_j\}$
Output: corrected labels y

for each attack window $\mathcal{I}_j$ **do**
  $v \leftarrow$ mode of $\mathrm{dst}(p)$ over SYN-only packets in $\mathcal{I}_j$
  if no SYN-only packets **then**
    $v \leftarrow$ mode of $\mathrm{src}(p)$ over RST packets
  end if
  if still undefined **then**
    $v \leftarrow$ mode of $\mathrm{dst}(p)$ over TCP/UDP packets
  end if
  for each $p \in \mathcal{I}_j$ **do**
    if $\mathrm{src}(p) = v$ **then**
      $y(p) \leftarrow 0$ ▷ victim reply
    else if $\mathrm{proto}(p) \notin \{\mathrm{TCP}, \mathrm{UDP}, \mathrm{QUIC}\}$ **then**
      $y(p) \leftarrow 0$ ▷ background
    else if $\mathrm{proto}(p) = \mathrm{QUIC} \wedge \mathrm{dst}(p) \neq v$ **then**
      $y(p) \leftarrow 0$
    end if
  end for
end for

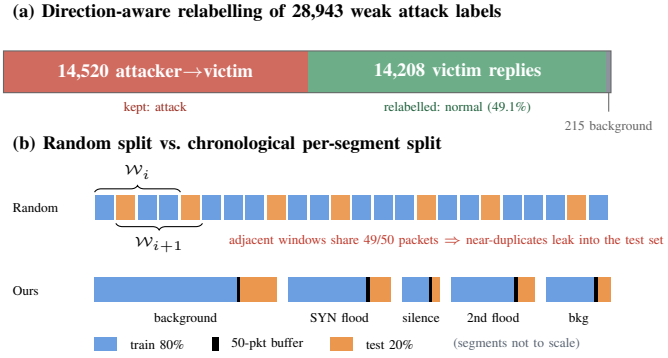


Fig. 4. Data conditioning. (a) Nearly half of the weak attack labels belonged to the victim's own replies. (b) Rolling windows overlap by 49 of 50 packets, so random splitting leaks test windows into training. FUZZYFORENSIOT splits every traffic segment chronologically and drops a 50-packet buffer at each boundary.

D. Leakage-Free Chronological Split

All features are rolling-window statistics, so neighbouring rows overlap almost completely:

$$|\mathcal{W}_i \cap \mathcal{W}_{i+k}| = W - k \quad (0 \le k < W). \tag{3}$$

A random row split therefore places near-duplicates of each test row in the training set. With such a split, an unpruned plain tree reached 88.45% accuracy, against 86.39% under the honest protocol (Section VIII-D). We instead split *each contiguous traffic segment* chronologically. The first 80% of every segment is used for training, the last 20% is held out, and the 50 packets at the boundary are dropped (Fig. 4b). Because the features look only backwards, no training window can contain a held-out packet.

TABLE III
THE 22 ROLLING-WINDOW FEATURES ($W = 50$)

Group	Features
Flood semantics (9)	SYN/ACK ratio, half-open count, same-source frequency, SYN density, UDP density, packet density, byte density, unique destination ports, mean inter-SYN time
Packet statistics (8)	packet length, inter-arrival Δt, SYN-only flag, length mean, length std., UDP ratio, SYN ratio, retransmission rate
Direction (5)	dst port < 1024, src port < 1024, RST flag, ICMP flag, response direction d_i

V. Proposed Method: Fuzzy Forensic Reasoning

A. Transport-Layer Feature Engineering

Twenty-two features are computed per window (Table III). They are grouped into flood semantics, packet statistics and per-packet direction. With S_i, A_i and H_i the numbers of SYN, ACK and SYN-only packets in $\mathcal{W}_i$:

$$r_i = \frac{S_i}{A_i + 1}, \tag{4}$$

$$H_i = \textstyle\sum_{p \in \mathcal{W}_i} \mathbf{1}[\mathrm{SYN}(p) \wedge \neg \mathrm{ACK}(p)], \tag{5}$$

$$\Delta t_i = \max\big(t_i - t_{i-W+1},\, \epsilon_i\big), \tag{6}$$

$$\epsilon_i = \begin{cases} 1\,\mathrm{s}, & i < W \ \text{(window filling)} \\ 10^{-3}\,\mathrm{s}, & i \geq W \end{cases} \tag{7}$$

$$\rho_i^{\mathrm{SYN}} = \min\Big(\frac{S_i}{\Delta t_i}, 10^3\Big), \quad \rho_i^{\mathrm{pkt}} = \min\Big(\frac{W}{\Delta t_i}, 2{\cdot}10^3\Big), \tag{8}$$

$$f_i = \frac{1}{|\mathcal{W}_i|} \textstyle\sum_{p \in \mathcal{W}_i} \mathbf{1}[\mathrm{src}(p) = \mathrm{src}(p_i)], \tag{9}$$

$$d_i = \mathbf{1}\big[\mathrm{sport}(p_i) < 1024 \ \vee \ \mathrm{RST}(p_i) \ \vee \ \mathrm{ICMP}(p_i)\big]. \tag{10}$$

The SYN/ACK ratio r_i and the half-open count H_i automate the SYN versus SYN-ACK comparison of [1]. The densities ρ quantify the I/O-graph spikes. The source frequency f_i captures single-source versus distributed origins. The warm-up floor $\epsilon_i = 1$ s prevents artificial density spikes while the window is still filling. The response-direction flag d_i is the key addition. Floods travel *to* a service port from an ephemeral port, while the victim's replies originate *from* the service port or are RST/ICMP control packets. Direction separates attacker from victim *inside the same burst*, which window densities alone cannot do.

B. Data-Driven Fuzzification

Ten attributes enter the fuzzy layer: the nine flood-semantic features and d_i. Each attribute is first normalised with training-set percentiles,

$$\tilde{x} = \min\Big(1, \max\Big(0, \frac{x - q_{0.01}}{q_{0.99} - q_{0.01}}\Big)\Big), \tag{11}$$

and then mapped to three triangular sets

$$\mu(\tilde{x}; a, b, c) = \max\Big(0, \min\Big(\frac{\tilde{x} - a}{b - a}, \frac{c - \tilde{x}}{c - b}\Big)\Big), \tag{12}$$

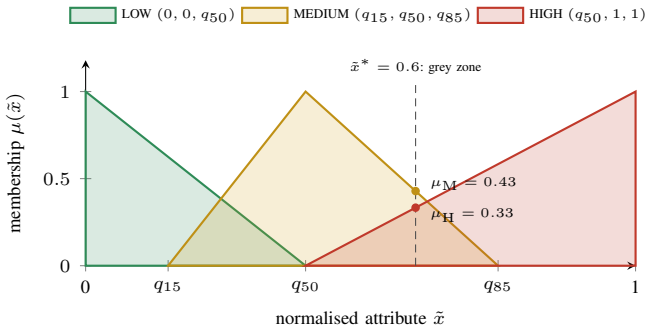


Fig. 5. Data-driven triangular membership functions placed at training percentiles (illustrative q values). A borderline window ($\tilde{x}^* = 0.6$) is simultaneously MEDIUM (0.43) and HIGH (0.33) instead of being forced to one side of a crisp threshold.

with the convention that a degenerate side ($a = b$ or $b = c$) is a step. Instead of hand-tuning the breakpoints, as in our early prototype (e.g., SYN-ACK/SYN ratio LOW$= [0, 0, 0.15]$), we place them at the 15th, 50th and 85th percentiles of the training traffic:

$$\begin{aligned} &\mathrm{LOW} = (0,\, 0,\, q_{50}), \quad \mathrm{MEDIUM} = (q_{15},\, q_{50},\, q_{85}), \\ &\mathrm{HIGH} = (q_{50},\, 1,\, 1). \end{aligned} \tag{13}$$

This yields 30 membership grades (Fig. 5). The sets adapt to each network's own traffic distribution and leave no constants to defend. Overlapping sets let a borderline value be partly MEDIUM and partly HIGH, which captures the "grey zone" between congestion and low-rate flooding that crisp thresholds cannot represent. For the binary direction flag, the sets collapse to crisp steps, which is exactly the behaviour a gate needs.

C. Direction-Gated Expert Rule Base

Eight Mamdani-style rules encode the flood signatures of Section II. Each rule is the min t-norm of two (R8: three) antecedent grades, gated by the LOW grade of the response direction:

$$R_k = \min\big(\mu_{A_k},\, \mu_{B_k},\, \mu_d^{\mathrm{LOW}}\big), \qquad k = 1, \ldots, 8. \tag{14}$$

The gate means that a victim's reply ($d_i = 1$, so $\mu_d^{\mathrm{LOW}} = 0$) can never fire an attack rule, which guarantees attacker-direction attribution (goal G4). Table IV lists the rules together with how often each one fires on the held-out data. The rule evidence is summarised in a fuzzy confidence that blends weighted evidence with the strongest single rule:

$$c = 0.55 \sum_{k=1}^{8} w_k R_k + 0.45 \max_k R_k, \qquad \textstyle\sum_k w_k = 1. \tag{15}$$

D. Lightweight Fuzzy Decision Tree

The tree sees *only* the fuzzy space,

$$\mathbf{z}_i = \big[\, \boldsymbol{\mu}_i\ (30),\ \mathbf{R}_i\ (8),\ c_i\ (1) \,\big] \in [0,1]^{39}. \tag{16}$$

Every split therefore reads as a linguistic condition, e.g., "μ^{HIGH}(byte density) > 0.18". Every root-to-leaf path becomes an IF-THEN rule, as in Fuzzy-ID3 [8], [17]. Classical

TABLE IV
DIRECTION-GATED RULE BASE: WEIGHTS AND HELD-OUT FIRING RATES ($R_k > 0.1$) ON ATTACK VS. NORMAL PACKETS

Rule	Antecedent ($\wedge\, d$ is LOW)	w_k	Atk.	Norm.
R1	SYN/ACK HIGH $\wedge$ half-open HIGH	0.16	63.9%	15.2%
R2	SYN density HIGH $\wedge$ inter-SYN LOW	0.15	62.3%	11.8%
R3	src frequency LOW $\wedge$ SYN/ACK HIGH	0.12	0.0%	0.0%
R4	SYN density HIGH $\wedge$ half-open HIGH	0.10	56.8%	10.9%
R5	dst-port diversity LOW $\wedge$ SYN density HIGH	0.10	11.8%	0.3%
R6	UDP density HIGH $\wedge$ pkt density HIGH	0.15	0.0%	7.7%
R7	byte density HIGH $\wedge$ pkt density HIGH	0.12	21.4%	3.0%
R8	UDP, SYN/ACK, pkt density all MEDIUM	0.10	8.6%	2.4%
c	fuzzy confidence, Eq. (15)	–	72.5%	25.3%

Algorithm 2 Shared lightweight tree training and rule fusion

Input: training features Z (fuzzy) or X (raw); labels y; fuzzy confidence c

Output: pruned tree T; fusion weight λ; threshold τ

$\mathcal{G} \leftarrow \{4,6,8,10,12\} \times \{5,20,40\} \times \{1{:}2, 1{:}3, 1{:}4, \text{bal.}\}$
$g^\star \leftarrow \arg\max_{g\in\mathcal{G}} \mathrm{F1}_{\mathrm{CV}}(g)$ ▷ 3-fold stratified
$T_0 \leftarrow \textsc{Fit}(g^\star)$; compute CCP path $\{\alpha_j\}$
$T \leftarrow$ smallest T_{α_j} with $\mathrm{F1}_{\mathrm{CV}} \geq \mathrm{F1}_{\mathrm{CV}}(g^\star) - 0.005$
for $\lambda \in \{0, 0.3, 0.5, 0.6, 0.7, 0.8, 1\}$ **do**
    $s \leftarrow \max(p_T(Z), \lambda c)$
    $(\tau_\lambda, J_\lambda) \leftarrow \arg\max_\tau J(\tau; s, y)$ s.t. recall ≥ 0.35
end for
return $T,\ \lambda^\star = \arg\max_\lambda J_\lambda,\ \tau_{\lambda^\star}$

Fuzzy-ID3 propagates membership degrees through a fuzzy-entropy criterion. We instead run crisp CART induction [16] over the fuzzified features. This keeps inference to at most nine comparisons and lets the baseline tree be trained by *exactly* the same procedure (Algorithm 2). A grid of 60 configurations is scored by 3-fold stratified cross-validated F1 on the training set. Cost-complexity pruning, which minimises $R_\alpha(T) = R(T) + \alpha|\tilde{T}|$, then selects the *smallest* tree whose CV-F1 lies within $\delta = 0.005$ of the best:

$$\alpha^\star = \arg\min_\alpha \ |T_\alpha| \quad \text{s.t.} \quad \mathrm{F1}_{\mathrm{CV}}(T_\alpha) \geq \mathrm{F1}^\star_{\mathrm{CV}} - \delta. \tag{17}$$

The selected configuration (depth 10, minimum leaf 5, class weight 1:4) is pruned with $\alpha^\star = 1.67 \times 10^{-4}$ from 371 to **89 nodes**. The result has depth 9 and 45 leaves and uses 20 of the 39 fuzzy features.

E. Rule Fusion and Decision

The deployed threat score fuses the tree's posterior with the rule evidence,

$$s = \max\big(p_T(\mathbf{z}),\ \lambda\, c\big), \tag{18}$$

so that windows where the tree is uncertain but expert rules fire strongly can still be escalated. The pair (λ, τ) maximises a single operating-point objective on the *training* predictions:

$$J(\tau) = 0.5\,\mathrm{F1} + 0.3\,\mathrm{Acc} + 0.2\,\mathrm{BAcc}, \quad \text{recall} \geq 0.35, \tag{19}$$

which gives $\lambda = 0.7$ and $\tau = 0.48$. Every baseline model is thresholded with the same objective.

F. Forensic Rule Extraction

Following Ye [8], every root-to-leaf path is exported as an IF-THEN rule whose confidence is the leaf purity. The deployed tree yields 45 rules. The 16 attack rules have a mean length of 6.8 conditions (6 to 9). Fig. 6 summarises the reasoning chain, and Section VIII-F traces a real held-out verdict.

VI. EDGE DEPLOYMENT AND GRADUATED RESPONSE

A. Live Detection Engine

The research pipeline runs on a laptop. The Raspberry Pi only loads the artifact, which contains the pruned tree, the feature order, the clip bounds, the membership parameters, λ, τ and W. The engine (`live_detector.py`) runs a scapy [25] sniffer thread with a BPF filter (e.g., `ip and not port 22`) that feeds a bounded queue, so a flood cannot exhaust memory. A worker thread maintains the 50-packet window. It recomputes the 22 features in pure Python (≈1 ms per window, without pandas) using *the same arithmetic as training*, which avoids train/serve skew. It then applies the frozen fuzzy layer and scores the window with Eq. (18). No verdicts are issued until 20 packets have been seen (warm-up). A deterministic replay mode (`--replay-csv`) supports reproducible tests.

B. Graduated, Reversible Response

Each score is attributed to the dominant source of the window, and a per-source state machine (Fig. 7) is updated:

$$a(s) = \begin{cases} \textsc{Block}, & s \geq \tau \\ \textsc{Throttle}, & \tau/2 \leq s < \tau \\ \textsc{Allow}, & s < \tau/2. \end{cases} \tag{20}$$

THROTTLE issues a rate-limit advisory with a 20 s TTL, for low-level floods that warrant active monitoring. BLOCK installs an `iptables` DROP rule (with `--live-firewall`; otherwise the block is simulated and logged) with a 120 s TTL and automatic release. This graduated design follows the analyst-triage practice of escalating HIGH threats while only monitoring MEDIUM ones. It keeps a single false alarm from becoming a permanent outage.

C. Safety Rails and Forensic Logging

The SSH client address (taken from `SSH_CLIENT`), loopback, broadcast and operator-supplied `--protect` addresses are never blocked. Direction gating ensures that the Pi's own SYN-ACKs, RSTs and ICMP errors cannot trigger mitigation. Each decision is appended to a JSON-lines event log with the timestamp, score, tree probability, fuzzy confidence, all eight rule activations, the window statistics, the dominant source and the action taken. Packet events are rate-limited so that a

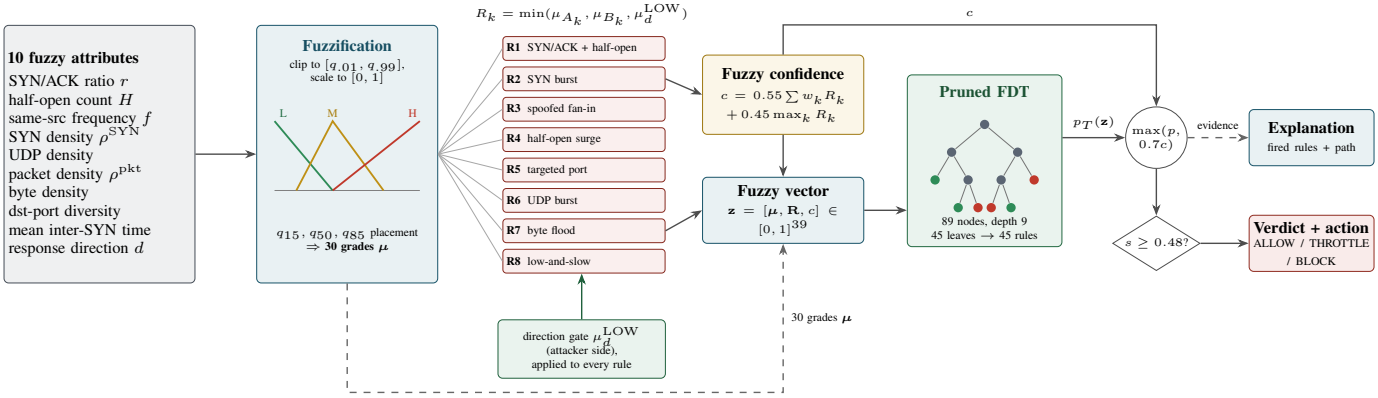
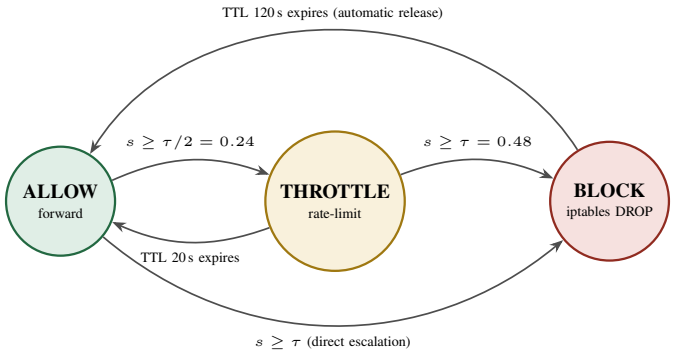


Fig. 6. Fuzzy reasoning pipeline of FUZZYFORENSIOT. Normalised attributes are fuzzified into 30 grades. Eight direction-gated rules (min t-norm) produce named attack evidence that is summarised by the fuzzy confidence c. A pruned tree trained only on the 39-dimensional fuzzy vector gives p_T, and the fused score $\max(p_T, 0.7c)$ selects the response. The fired rules and the decision path form the explanation.



Safety rails: SSH client, loopback, broadcast and operator-protected IPs never leave ALLOW; direction gating stops the Pi's own replies, RSTs and ICMP errors from triggering mitigation.

Fig. 7. Per-source graduated response. Escalation follows the fused threat score, and every mitigation expires automatically after its TTL.

flood cannot inflate the log. An atomic state snapshot is written every second. Together these provide a time-ordered evidence trail for the preservation and presentation stages of the forensic process model [1], [8].

D. Dashboard and Traffic Simulator

A dependency-free dashboard (Python standard-library HTTP) runs on the Pi. It shows KPI cards, the live packet stream, a per-IP threat board with manual block/release controls, score timelines, the fired-rule explanation of each decision, CPU/RAM/temperature telemetry and the benchmark table. A companion simulator generates *normal*, *suspicious* (low-and-slow scan), *low-flood*, *congestion*, *syn-flood* and *udp-flood* traffic, plus a scripted *story* mode that logs ground-truth timestamps for time-to-detect scoring. The congestion scenario uses complete TCP handshakes at a high rate and therefore serves as a built-in false-positive test.

VII. EXPERIMENTAL SETUP

Models. All models are evaluated on the same chronological hold-out (38,747 packets, 2,962 attacks) and thresholded with the same objective, Eq. (19), tuned on the training data only.

- **Fuzzy DT (proposed, deployed):** 39 fuzzy features, Algorithm 2, fused score with $\lambda = 0.7$.
- **Plain DT:** the 22 raw features, trained by the *identical* Algorithm 2. It selects the same configuration and is pruned from 463 to 71 nodes. The only difference from the proposed model is the feature space.
- **Deep MLP:** hidden layers 256-128-64-32 with ReLU, Adam (learning rate 5×10^{-4}, adaptive), L2 penalty 10^{-4}, batch size 256, early stopping (15% validation data, patience 25), standardised inputs and attack oversampling to 1:2.
- **Fuzzy Forest (reference):** 30 trees [26] (depth 14, balanced subsampling) on the same 39 fuzzy features, fused with $\lambda = 1$.

Metrics. We report accuracy, balanced accuracy (BAcc), precision, recall and F1 for the attack class, ROC-AUC computed on each model's probability output, false-positive rate (FPR), serialised model size, and per-packet latency. Latency is the median of 5 vectorised runs over 2,000 held-out packets on the real artifacts, including the fuzzification step. **Software.** Python 3.12, scikit-learn [27], NumPy, pandas, joblib and scapy [25]. Training is done on a laptop, and inference runs on the Raspberry Pi under Raspberry Pi OS.

VIII. RESULTS AND DISCUSSION

A. Overall Comparison

Table V and Fig. 8 summarise the hold-out results. The deployed Fuzzy DT achieves the best accuracy among the single models (87.74%), the best precision (34.51%) and the best F1 of all four models (0.456). It improves on the identically trained plain DT by $+1.35$ accuracy points, $+2.54$ precision points and $+0.019$ F1. It also beats the deep MLP on accuracy, precision and F1 while being $125\times$ smaller (9.1 KB vs. 1.14 MB). The Fuzzy Forest gains $+0.55$ accuracy points over the single tree but loses 0.014 F1 and 6.6 recall points, at roughly $300\times$ the size (35,478 vs. 89 nodes). The single pruned tree is therefore the right choice for the edge.

The plain DT has the highest ROC-AUC (0.917). The Fuzzy DT's tree probability scores 0.908, and its deployed fused score scores 0.911. All models are thus close in ranking quality. The fuzzy representation pays off at the *operating point*: at the tuned threshold it converts the same ranking ability into fewer false alarms.

B. False Alarms and Per-Segment Behaviour

Table VI lists the confusion counts. The Fuzzy DT raises 3,781 false alarms against 4,364 for the plain DT (583 fewer, -13.4%) and 4,322 for the MLP (541 fewer). Fewer false alarms means less alert fatigue for the analyst and fewer wrongful blocks. All 3,781 false positives fall in the background segments, and *none* fall on the victim's replies inside the attack phases. This is direct evidence that direction gating works. Recall is 98.4% in the second flood segment but 55.5% in the SYN-flood segment, which is the main target for improvement. Operationally, a source is blocked as soon as one of its windows crosses τ, so a flood does not need to be detected packet by packet.

C. Size, Accuracy and Latency

Fig. 9 plots F1 against model size. The Fuzzy DT is the only model in the top-left (small and accurate) corner. Its per-packet cost of 13.4 μs is dominated by fuzzification. This is slower than the vectorised plain DT (0.76 μs) and MLP (5.05 μs), but it still sustains about 75,000 packets/s on one core. That is almost two orders of magnitude below the 1 ms inter-arrival time of the $\approx$1,000 pps floods in our capture, so the edge budget (G1) is comfortably met. Training-time pruning helped accuracy as well as size: the unpruned 371-node fuzzy tree scored 87.16%, while the 89-node tree scores 87.74%. On rolling-window data, regularisation rather than capacity drives generalisation.

D. Ablation Study

Table VII isolates the components. (i) The *expert rules alone*, thresholding c with its own tuned threshold, behave like a hand-written fuzzy IDS. They reach a recall of 72.5% but an FPR of 25.3% and an F1 of only 0.303, which confirms that fixed rules are too blunt. (ii) Learning a pruned tree on top of the fuzzy space *halves* the false-positive rate and lifts F1 by 0.149. (iii) Fusing the rules back in (Eq. (18)) recovers 22 more attacks with *zero* additional false alarms. These are windows where the tree is uncertain but expert evidence is strong. (iv) With learner, grid, pruning and threshold held fixed, replacing the raw features with fuzzy ones accounts for the full gain over the plain DT. The improvement therefore comes from the representation, not from tuning. (v) Evaluation hygiene matters: a random split inflates the plain DT by 2.06 points, and training on the uncorrected weak labels depresses ROC-AUC to about 0.79. (vi) Data-driven membership placement matches hand-placed sets (train CV-F1 0.383 vs. 0.380) while removing every hand-tuned constant.

E. What the Tree Learned

Fig. 10 shows the impurity-based feature importances of the deployed tree. The *root* split is "$c \leq 0.005$", i.e., whether any direction-gated rule fired at all. The fuzzy confidence alone carries 35.4% of the importance, and together with the direction grade more than half. The tree thus learned to *trust and refine* the expert rules rather than replace them. The firing statistics in Table IV show this as well. R1, R2 and R4 fire on 57–64% of attack packets but only 11–15% of normal packets. R5 (a single targeted port under a SYN burst) is the most specific rule (11.8% vs. 0.3%). R3 never fired because the captured attack came from one host, so source-dispersion evidence was absent. R6 fired only on background UDP/QUIC bursts, which explains part of the residual false alarms and motivates per-network recalibration.

F. Case Study: Anatomy of a Forensic Verdict

Fig. 11 traces a real held-out packet from the SYN-flood phase through the deployed tree. The raw window shows a SYN to port 80 with TCP window 512 and zero payload, 38 half-open connections out of 50 packets, saturated SYN and packet densities, and a 0.18 ms mean inter-SYN gap. Rules R1 and R4 fire fully, R2 at 0.84, R5 at 0.50 and R7 at 0.31, giving $c = 0.71$. The tree path (six conditions) ends in a pure attack leaf supported by 53 training windows. The fused score is $s = \max(1.0, 0.7 \times 0.71) = 1.0 \geq \tau$, so the source is blocked. The exported rule reads like an analyst's note:

```
IF fuzzy evidence present (c=0.71)
  AND SYN/ACK ratio is NOT low (μ=0.00)
  AND SYN density is NOT medium (μ=0.00)
  AND byte density is HIGH (μ=0.31)
  AND SYN-burst rule R2 holds (μ=0.84)
  AND dst-port diversity is LOW (μ=0.50)
THEN TCP-SYN flood [leaf purity 100%, n=53]
ACTION: BLOCK src, iptables DROP, TTL 120 s
```

An equally accurate MLP would offer only a probability. Here every term maps to an indicator a forensic examiner already uses [1].

G. From Manual Indicators to Automated Evidence

Table VIII maps each indicator used in the manual Wireshark investigations [1], [6] to its automated counterpart. Most indicators become continuous fuzzy evidence. ICMP-unreachable traffic is deliberately used as a *direction cue* (victim-side control traffic) rather than as attack evidence. This keeps the victim's own error messages from being blamed on it.

H. Edge Feasibility

On the Raspberry Pi the deployed path consists of pure-Python feature extraction ($\approx$1 ms per window) followed by the fuzzy layer and the 89-node tree. During the live demonstration the dashboard telemetry showed about 23% CPU, 41% RAM and a SoC temperature of about 52 °C. In the scripted live scenarios, the *congestion* phase (legitimate handshakes at a high rate) stayed in ALLOW because direction gating ignored the Pi's replies. During the *SYN-flood* phase the score saturated at 1.0, the source was escalated to BLOCK, and the dashboard

TABLE V

HOLD-OUT RESULTS (38,747 PACKETS, 7.64% ATTACK). BEST VALUE PER COLUMN IN BOLD. LATENCY IS PER PACKET AND INCLUDES FUZZIFICATION

Model	Feature space	Acc. (%)	BAcc (%)	Prec. (%)	Recall (%)	F1	AUC	FPR (%)	Size	Latency per pkt
Fuzzy DT (proposed, deployed)	39 fuzzy	87.74	78.34	34.51	67.25	**0.456**	0.908	10.57	9.1 KB	13.4 μs
Fuzzy Forest, 30 trees (reference)	39 fuzzy	**88.29**	75.64	**34.78**	60.70	0.442	0.912	**9.42**	2.72 MB	24.0 μs
Plain DT (identical protocol)	22 raw	86.39	78.52	31.97	69.24	0.438	**0.917**	12.20	7.1 KB	**0.76 μs**
Deep MLP (256-128-64-32)	22 raw	86.51	**78.67**	32.24	**69.41**	0.440	0.911	12.08	1.14 MB	5.05 μs

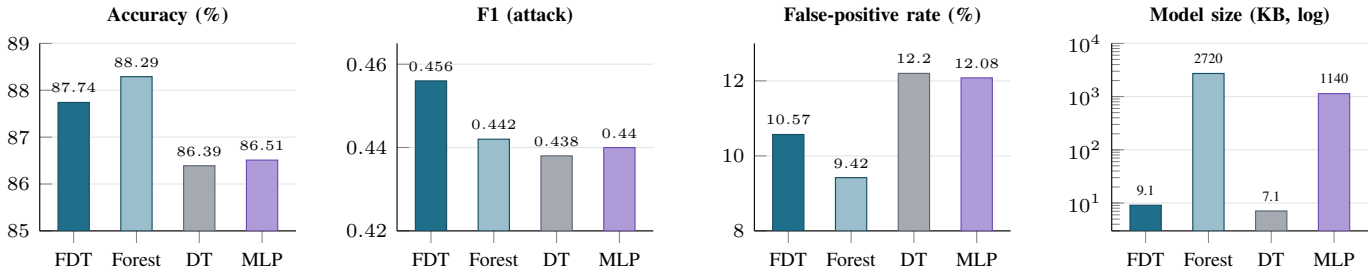


Fig. 8. Hold-out comparison. The 9.1 KB Fuzzy DT has the best F1, cuts false positives relative to both raw-feature baselines, and is 125× smaller than the deep MLP and 300× smaller than the Fuzzy Forest.

TABLE VI

CONFUSION COUNTS ON THE HOLD-OUT (2,962 ATTACK / 35,785 NORMAL)

Model	TP	FN	FP	TN	FPR
Fuzzy DT	1,992	970	**3,781**	32,004	10.57%
Fuzzy Forest	1,798	1,164	3,372	32,413	9.42%
Plain DT	2,051	911	4,364	31,421	12.20%
Deep MLP	2,056	906	4,322	31,463	12.08%

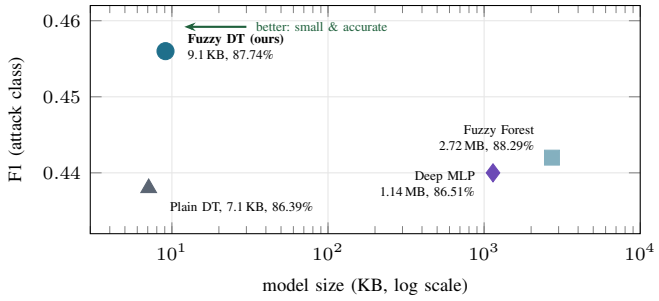


Fig. 9. F1 versus model size. The proposed tree dominates both raw-feature baselines and matches or beats models two to three orders of magnitude larger.

TABLE VII

ABLATIONS ON THE HOLD-OUT SET (ACC. IN %)

Variant	Acc.	F1	Recall	FPR
Rules only ($c \geq 0.09$)	74.50	0.303	72.52	25.33
Fuzzy tree only ($\lambda = 0$)	87.68	0.452	66.51	10.57
Fuzzy tree + rule fusion ($\lambda = 0.7$)	**87.74**	**0.456**	67.25	10.57
Unpruned fuzzy tree (371 nodes)	87.16	–	–	–
Plain DT, raw features (honest split)	86.39	0.438	69.24	12.20
Plain DT, *random* split (leaky)	88.45	–	–	–

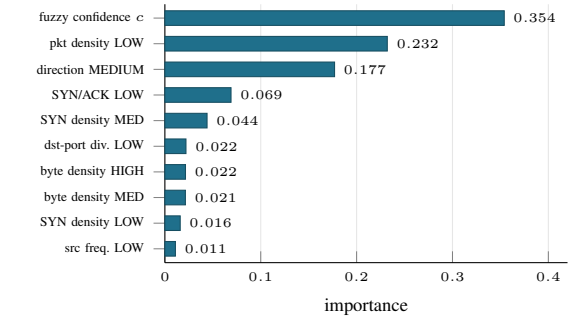


Fig. 10. Top-10 feature importances of the deployed Fuzzy DT. The rule-layer output (fuzzy confidence) is the root split and the most important feature.

displayed the R1/R2 explanation ("high SYN burst paired with a low SYN-ACK ratio"). Blocked sources were released automatically after their TTL.

IX. LIMITATIONS AND THREATS TO VALIDITY

Single capture. The results come from one lab capture with two attack phases (about 50 s of flooding) from a single attacker host, against a large and diverse background. The spoofing option did not produce dispersed sources in the capture, so R3 was never exercised. Generalisation to other networks and to public benchmarks such as CICIoT2023 [23] and Bot-IoT [13] remains to be shown. **Second flood phase.** Although the second phase was generated with the UDP-flood command, the attacker-to-victim packets labelled in it are TCP control traffic (RST/PSH-ACK). The static dataset's attack class is therefore TCP-centric. UDP-specific rules (R6, R8) were exercised only through the live simulator. **Packet-level precision.** Precision is 34.5% at packet granularity on a 13:1 imbalanced hold-out. Operationally, alerts are aggregated per source and per TTL, so

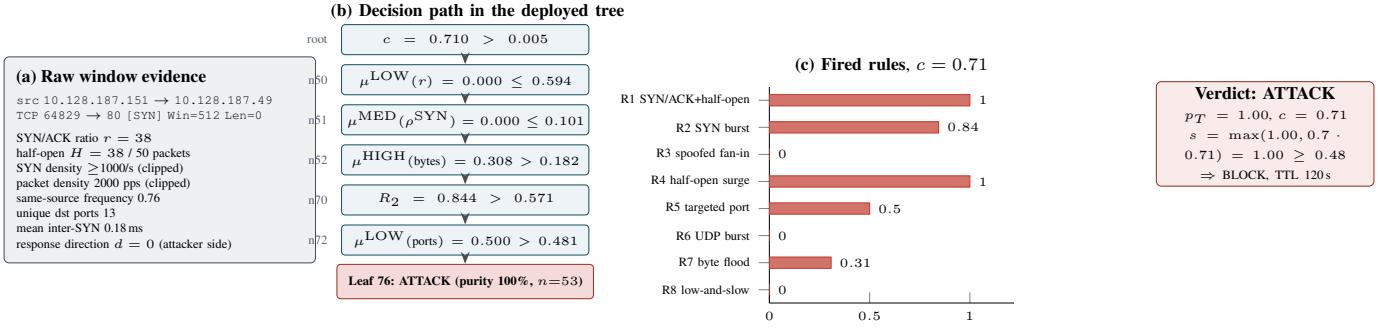


Fig. 11. Anatomy of a real held-out verdict. (a) Transport-layer evidence of the window. (b) The six-condition path through the 89-node tree, ending in a pure attack leaf. (c) Activations of the eight direction-gated rules. The explanation is produced by the same fuzzy reasoning that produced the verdict; no post-hoc explainer is needed.

TABLE VIII
MANUAL FORENSIC INDICATORS [1], [6] AND THEIR AUTOMATED COUNTERPARTS

Manual indicator (Wireshark)	**FUZZYFORENSIOT counterpart**
SYN vs. SYN-ACK counts (`tcp.flags` filters)	SYN/ACK ratio r, half-open H; rules R1, R3, R4
I/O-graph spikes	SYN/packet/byte densities; R2, R4, R7
Spoofed or repeated source IPs	same-source frequency f; R3
Single targeted service port	dst-port diversity; R5
UDP to port 80, UDP volume	UDP density, dst-port class; R6, R8
ICMP host unreachable (`icmp.type==3`)	ICMP flag in direction gate d
Packet-length bins (40–2,599 B)	length mean / std. per window
Out-of-order, retransmissions	retransmission rate
Analyst's binary judgement	graded score s + IF-THEN rule + action

one blocked host counts once. Window- or flow-level alerting would lower the alert volume further. **Skewed attributes.** Percentile placement collapses to crisp steps for heavily skewed attributes. For example, the median UDP density in training is zero. **Live features.** The retransmission feature relies on Wireshark annotations and is close to zero in live scapy mode. The live-edge observations are qualitative, and a systematic time-to-detect benchmark on the Pi is still pending. **Induction method.** We use crisp CART over fuzzy features rather than native fuzzy-entropy ID3 with soft membership propagation.

X. CONCLUSION AND FUTURE WORK

This paper presented FUZZYFORENSIOT, which automates the transport-layer indicators of manual IoT network forensics [1], [6] with the fuzzy decision-tree reasoning proposed for forensic analysis [8], and deploys the result on a Raspberry Pi. Data-driven membership functions, eight direction-gated expert rules and a pruned decision tree trained only on the fuzzy space form a 9.1 KB model. Under a leakage-free protocol it reaches 87.74% accuracy and an F1 of 0.456. It beats an identically trained plain tree and a 125× larger deep MLP, and raises 583 fewer false alarms than the plain tree. Every verdict carries a human-readable IF-THEN explanation and a graduated, reversible response. Equally important, we showed that weak labels (49.1% victim replies) and random splits of rolling-window data can each distort IDS evaluation by several points, and we provided simple fixes for both.

Future work will (i) evaluate cross-dataset generalisation on CICIoT2023 and Bot-IoT and capture multi-host, genuinely spoofed and UDP-dominant floods; (ii) implement native Fuzzy-ID3 induction with fuzzy entropy and soft path propagation and compare it with the crisp-over-fuzzy tree; (iii) add the remaining indicators from the manual studies, namely TCP window-size anomaly (e.g., Win=256), Shannon source-IP entropy, ICMP-unreachable rate and a 100 ms temporal burst score; (iv) estimate retransmissions from sequence numbers so the feature works in live mode; (v) adapt membership placement online with streaming quantiles; (vi) extend the rule base to application-layer IoT floods (HTTP, MQTT); and (vii) port inference to C/ONNX for microcontroller-class devices, together with a quantitative on-device time-to-detect benchmark.

ACKNOWLEDGMENT

The authors thank the Indian Institute of Information Technology Sri City for the laboratory facilities and computing resources that supported this work.

REFERENCES

[1] S. K. Naing, S. T. Oo, H. W. Y. Oo, P. Y. Lin, K. M. Nwe, and M. M. S. Thwin, "IoT network forensics based on transport layer," in *Proc. IEEE Conf. Comput. Appl. (ICCA)*, Yangon, Myanmar, Feb. 2023, pp. 312–317, doi: 10.1109/ICCA51723.2023.10182142.

[2] M. Antonakakis *et al.*, "Understanding the Mirai botnet," in *Proc. 26th USENIX Security Symp.*, Vancouver, BC, Canada, 2017, pp. 1093–1110.

[3] C. Kolias, G. Kambourakis, A. Stavrou, and J. Voas, "DDoS in the IoT: Mirai and other botnets," *Computer*, vol. 50, no. 7, pp. 80–84, 2017.

[4] W. Eddy, "TCP SYN flooding attacks and common mitigations," IETF RFC 4987, Aug. 2007.

[5] CERT Coordination Center, "TCP SYN flooding and IP spoofing attacks," CERT Advisory CA-1996-21, Carnegie Mellon Univ., Pittsburgh, PA, USA, 1996.

[6] M. S. Khaing, Y. M. Thant, T. Tun, C. S. Htwe, and M. M. S. Thwin, "IoT botnet detection mechanism based on UDP protocol," in *Proc. IEEE Conf. Comput. Appl. (ICCA)*, Yangon, Myanmar, Feb. 2020.

[7] M. Stoyanova, Y. Nikoloudakis, S. Panagiotakis, E. Pallis, and E. K. Markakis, "A survey on the Internet of Things (IoT) forensics: Challenges, approaches, and open issues," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 2, pp. 1191–1221, 2nd Quart. 2020.

[8] J. Ye, "A fuzzy decision tree reasoning method for network forensics analysis," in *Proc. World Automation Congr. (WAC)*, San Antonio, TX, USA, Oct. 2022, pp. 41–45, doi: 10.23919/WAC55640.2022.9934389.

[9] R. Doshi, N. Apthorpe, and N. Feamster, "Machine learning DDoS detection for consumer Internet of Things devices," in *Proc. IEEE Security Privacy Workshops (SPW)*, San Francisco, CA, USA, 2018, pp. 29–35.

[10] Y. Meidan *et al.*, "N-BaIoT—Network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Comput.*, vol. 17, no. 3, pp. 12–22, Jul.–Sep. 2018.

[11] A. Adadi and M. Berrada, "Peeking inside the black-box: A survey on explainable artificial intelligence (XAI)," *IEEE Access*, vol. 6, pp. 52138–52160, 2018.

[12] S. M. Bridges and R. B. Vaughn, "Fuzzy data mining and genetic algorithms applied to intrusion detection," in *Proc. 23rd Nat. Inf. Syst. Security Conf. (NISSC)*, Baltimore, MD, USA, 2000.

[13] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Gener. Comput. Syst.*, vol. 100, pp. 779–796, 2019.

[14] L. A. Zadeh, "Fuzzy sets," *Inf. Control*, vol. 8, no. 3, pp. 338–353, 1965.

[15] J. R. Quinlan, "Induction of decision trees," *Mach. Learn.*, vol. 1, no. 1, pp. 81–106, 1986.

[16] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.

[17] Y. Yuan and M. J. Shaw, "Induction of fuzzy decision trees," *Fuzzy Sets Syst.*, vol. 69, no. 2, pp. 125–139, 1995.

[18] C. Z. Janikow, "Fuzzy decision trees: Issues and methods," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 28, no. 1, pp. 1–14, Feb. 1998.

[19] A. Dimolianis, A. Pavlidis, and V. Maglaris, "SYN flood attack detection and mitigation using machine learning traffic classification and programmable data plane filtering," in *Proc. 24th Conf. Innov. Clouds, Internet Netw. Workshops (ICIN)*, Paris, France, 2021, pp. 126–133.

[20] T.-M. Hoang, N.-H. Tran, V.-L. Thai, D.-L. Nguyen, and N.-H. Nguyen, "An efficient IDS using FIS to detect DDoS in IoT networks," in *Proc. 9th NAFOSTED Conf. Inf. Comput. Sci. (NICS)*, 2022.

[21] P. Keshavamurthy and S. Kulkarni, "A hybrid machine learning–fuzzy cognitive map approach for fast, reliable DDoS attack detection," in *Proc. Cyber Secur. Netw. Conf. (CSNet)*, 2024, pp. 228–235, doi: 10.1109/CSNET64211.2024.10851754.

[22] C. Kim, C. So-In, Y. Kongsorot, and P. Aimtongkham, "FLSec-RPL: A fuzzy logic-based intrusion detection scheme for securing RPL-based IoT networks against DIO neighbor suppression attacks," *Cybersecurity*, vol. 7, 2024, doi: 10.1186/s42400-024-00223-x.

[23] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, Art. no. 5941, 2023.

[24] S. Sanfilippo, "hping3: Active network security tool," [Online]. Available: http://www.hping.org

[25] P. Biondi and the Scapy community, "Scapy: Packet manipulation program and library," [Online]. Available: https://scapy.net

[26] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[27] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.